

Homework 06

Question 1: Compare KNN and logistic regression

Use [breast-cancer.csv](#), which contains 569 cases from the [Wisconsin Diagnostic Breast Cancer dataset](#). Predict y (1 for malignant and 0 for benign) using `mean_radius`, `mean_texture`, and `mean_smoothness`.

Set a random seed and randomly split the observations into 70% training and 30% test data. Use the same split for every model. Standardize the predictors using the training means and standard deviations, and apply the same transformations to the test data.

Fit ordinary logistic regression with an intercept and no penalty, and KNN with Euclidean distance, equal neighbor weights, and

$$k \in \{5, 15, 25, 35, 45\}.$$

Using the estimated probabilities for $y = 1$ on the test data, plot all six ROC curves in one figure and report their AUC values. Which of the six fitted models do you prefer? Give a brief justification based on your results.

We compare the models based on their estimated probabilities of malignancy rather than class labels from a single cutoff. For KNN, the estimated probability is the fraction of the k nearest training observations that are malignant:

$$\hat{p}(\mathbf{x}) = \frac{1}{k} \sum_{i \in N_k(\mathbf{x})} y_i,$$

where $N_k(\mathbf{x})$ contains those neighbors. Since the responses are coded as 0 and 1, their average gives this probability.

After splitting the data, calculate each predictor's mean and standard deviation using only the training set. Use these values to standardize both the training and test sets so the test data do not influence how distances are measured.

```
library(FNN)
```

```
Warning: package 'FNN' was built under R version 4.3.3
```

```
library(pROC)
```

```
Warning: package 'pROC' was built under R version 4.3.3
```

```
Type 'citation("pROC")' for a citation.
```

```
Attaching package: 'pROC'
```

```
The following objects are masked from 'package:stats':
```

```
cov, smooth, var
```

```

data_dir <- if (file.exists("data/breast-cancer.csv")) {
  "data"
} else {
  "practice/weeks/week-06/data"
}
data <- read.csv(file.path(data_dir, "breast-cancer.csv"))
set.seed(432)
train_id <- sort(sample(seq_len(nrow(data)), floor(0.7 * nrow(data))))
train <- data[train_id, ]
test <- data[-train_id, ]
predictors <- c("mean_radius", "mean_texture", "mean_smoothness")

x_bar <- colMeans(train[, predictors])
s <- sapply(train[, predictors], sd)
X_train <- scale(train[, predictors], center = x_bar, scale = s)
X_test <- scale(test[, predictors], center = x_bar, scale = s)
y_train <- train$y
y_test <- test$y

```

Fit an unpenalized logistic regression model and five KNN models. For KNN, R's `knn.reg` averages the neighbors' 0/1 responses, while Python's `predict_proba` returns the same proportions. The logistic regression model includes an intercept, and setting `C=np.inf` in Python removes the regularization penalty.

```

fit <- glm(
  y ~ .,
  data = data.frame(y = y_train, X_train),
  family = binomial()
)

k_values <- c(5, 15, 25, 35, 45)
model_names <- c("Logistic regression", paste("KNN, k =", k_values))

p_test <- matrix(
  NA_real_,
  nrow = nrow(X_test),
  ncol = length(model_names)
)
colnames(p_test) <- model_names

p_test[, 1] <- predict(
  fit,
  newdata = data.frame(X_test),
  type = "response"
)

for (m in seq_along(k_values)) {
  p_test[, m + 1] <- FNN::knn.reg(
    train = X_train,
    test = X_test,
    y = y_train,
    k = k_values[m],
    algorithm = "brute"
  )$pred
}

roc_test <- vector("list", length(model_names))
auc_test <- numeric(length(model_names))

```

```

for (m in seq_along(model_names)) {
  roc_test[[m]] <- pROC::roc(
    response = y_test,
    predictor = p_test[, m],
    levels = c(0, 1),
    direction = "<",
    quiet = TRUE
  )
  auc_test[m] <- as.numeric(pROC::auc(roc_test[[m]]))
}

data.frame(
  Model = model_names,
  Test_AUC = round(auc_test, 4)
)

```

	Model	Test_AUC
1	Logistic regression	0.9746
2	KNN, k = 5	0.9329
3	KNN, k = 15	0.9480
4	KNN, k = 25	0.9599
5	KNN, k = 35	0.9727
6	KNN, k = 45	0.9741

```

colors <- c(
  "#222222", "#0072B2", "#D55E00",
  "#009E73", "#CC79A7", "#806000"
)
line_types <- c(1, 2, 3, 4, 5, 6)

plot(
  c(0, 1), c(0, 1),
  type = "n",
  xaxs = "i",
  yaxs = "i",
  xlab = "False-positive rate",
  ylab = "True-positive rate"
)

abline(0, 1, col = "gray70", lty = 2)

for (m in seq_along(model_names)) {
  lines(
    1 - roc_test[[m]]$specificities,
    roc_test[[m]]$sensitivities,
    col = colors[m],
    lty = line_types[m],
    lwd = 2
  )
}

legend(
  "bottomright",
  legend = sprintf("%s: AUC = %.4f", model_names, auc_test),
  col = colors,
  lty = line_types,
  lwd = 2,

```

```
cex = 0.8,  
bty = "n"  
)
```

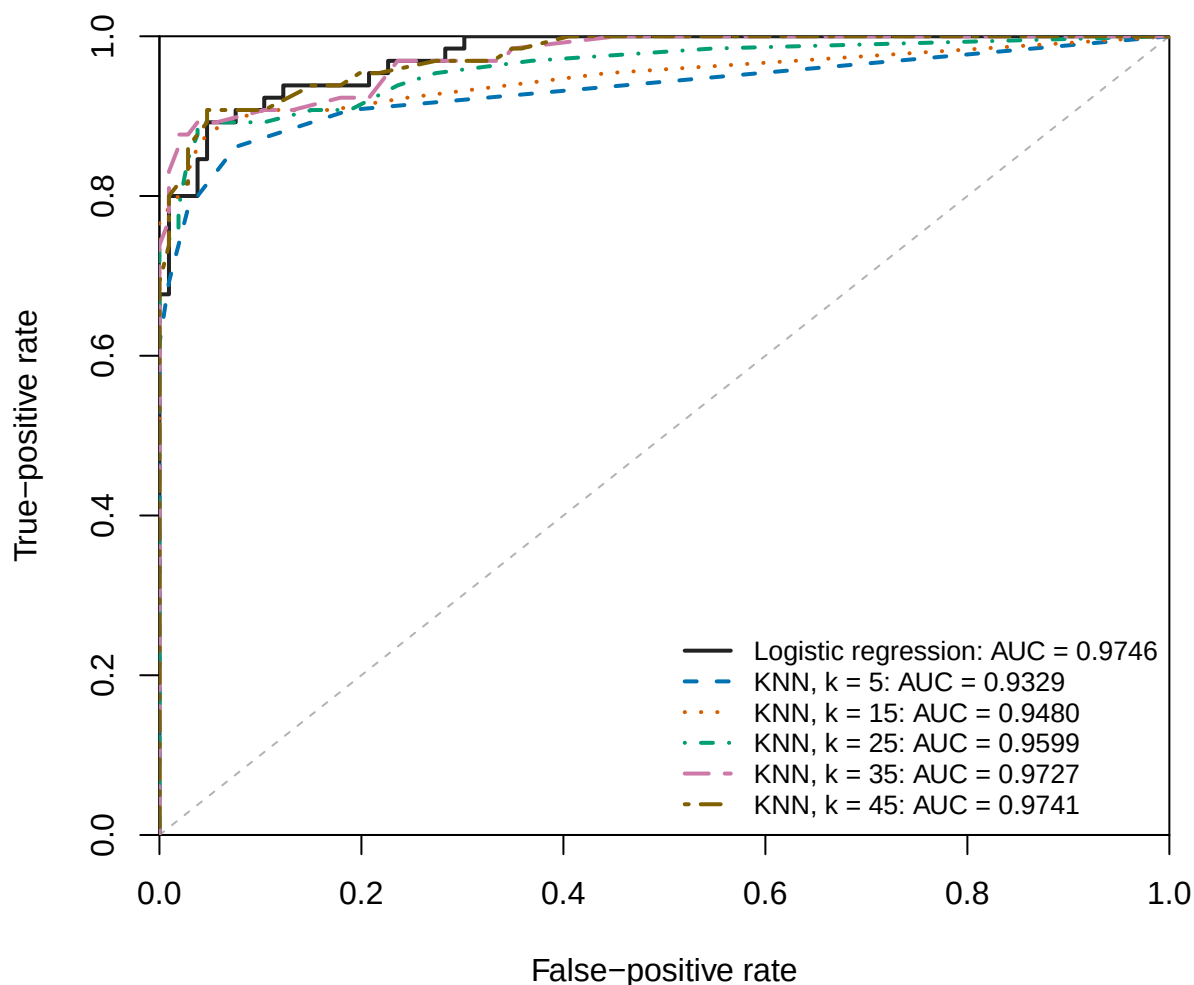


Figure 1: Test ROC curves for logistic regression and five KNN fits.

I would choose logistic regression for this split because it has the highest test AUC, 0.9746, and provides a simple fitted model. KNN with $k = 45$ is very close at 0.9741, followed by $k = 35$ at 0.9727. Smaller neighborhoods perform less well, with AUCs of 0.9329, 0.9480, and 0.9599 for $k = 5, 15, 25$. The ROC curves also suggest that larger neighborhoods rank these test cases better than $k = 5$.

These results come from just one test split, so the small differences do not mean one method will always perform better. AUC measures how well a model ranks cases, not the percentage it classifies correctly. For final model selection, I would use validation or cross-validation on the training data to choose the method and k , then evaluate the selected model on a fresh test set.

Question 2: Training and test ROC curves

Use the same data, training/test split, and standardization as in Question 1. Fit KNN with Euclidean distance, equal neighbor weights, and

$$k \in \{1, 3, 5, 10, 20\}.$$

Use each fitted model to estimate probabilities for both the training and test observations. For training predictions, include each observation itself among its k nearest neighbors.

For each k , use a separate panel to show the training and test ROC curves together. Report both AUC values.

Explain the difference between training and test performance and how it changes with k . In particular, explain what happens at $k = 1$ and why training performance may not reflect performance on new observations.

When $k = 1$, each training observation is its own nearest neighbor, so its predicted probability matches its observed response and the training AUC is 1. This perfect training result does not tell us how well the model will predict new observations, whose labels it has not seen.

Use the same data split and standardization as in Question 1. When generating training predictions, explicitly pass the training predictors so each observation can be included as its own neighbor.

```
k_values <- c(1, 3, 5, 10, 20)
p_train <- matrix(NA_real_, nrow(X_train), length(k_values))
p_test <- matrix(NA_real_, nrow(X_test), length(k_values))
roc_train <- roc_test <- vector("list", length(k_values))
auc_train <- auc_test <- numeric(length(k_values))
for (m in seq_along(k_values)) {
  k <- k_values[m]
  # test = X_train includes each training observation itself.
  p_train[, m] <- knn.reg(train = X_train, test = X_train,
    y = y_train, k = k, algorithm = "brute")$pred
  p_test[, m] <- knn.reg(train = X_train, test = X_test,
    y = y_train, k = k, algorithm = "brute")$pred
  roc_train[[m]] <- roc(y_train, p_train[, m], levels = c(0, 1),
    direction = "<", quiet = TRUE)
  roc_test[[m]] <- roc(y_test, p_test[, m], levels = c(0, 1),
    direction = "<", quiet = TRUE)
  auc_train[m] <- as.numeric(auc(roc_train[[m]]))
  auc_test[m] <- as.numeric(auc(roc_test[[m]]))
}
data.frame(k = k_values, Training_AUC = round(auc_train, 4),
  Test_AUC = round(auc_test, 4))
```

	k	Training_AUC	Test_AUC
1	1	1.0000	0.8764
2	3	0.9905	0.9138
3	5	0.9847	0.9329
4	10	0.9862	0.9446
5	20	0.9880	0.9586

```
old_par <- par(mfrow = c(2, 3), mar = c(4, 4, 2, 1))
for (m in seq_along(k_values)) {
  plot(1 - roc_train[[m]]$specificities, roc_train[[m]]$sensitivities,
    type = "l", col = "#0072B2", lwd = 2,
    xlim = c(0, 1), ylim = c(0, 1), xaxs = "i", yaxs = "i",
    main = paste("k =", k_values[m]),
    xlab = "False-positive rate", ylab = "True-positive rate")
  lines(1 - roc_test[[m]]$specificities, roc_test[[m]]$sensitivities,
    col = "#D55E00", lty = 2, lwd = 2)
  abline(0, 1, col = "gray70", lty = 3)
  legend("bottomright", c(sprintf("Training AUC = %.4f", auc_train[m]),
    sprintf("Test AUC = %.4f", auc_test[m])),
```

```
col = c("#0072B2", "#D55E00"), lty = c(1, 2), lwd = 2,
cex = 0.8, bty = "n")
}
plot.new()
```

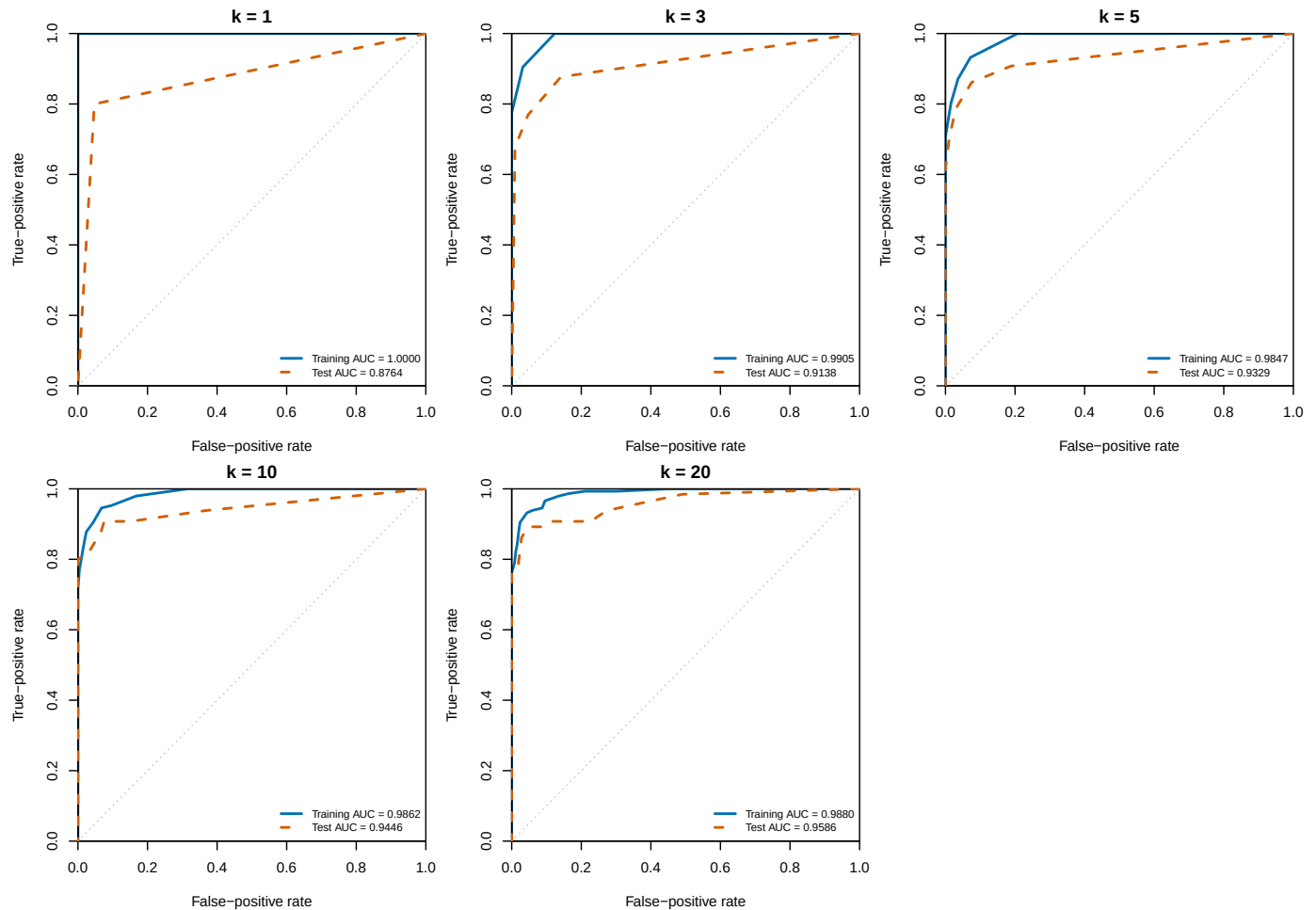


Figure 2: Training and test ROC curves for each KNN neighborhood size. Each training observation is included among its own neighbors.

```
par(old_par)
```

At $k = 1$, the training AUC is perfect at 1.0000, but the test AUC is only 0.8764. As k increases to 3, 5, 10, 20, test AUC improves to 0.9138, 0.9329, 0.9446, and 0.9586. The gap between training and test AUC also narrows, from about 0.124 to 0.029.

Training AUC does not consistently decrease: it drops to 0.9847 at $k = 5$, then increases to 0.9862 at $k = 10$ and 0.9880 at $k = 20$. Larger neighborhoods smooth the predicted probabilities, but they can still improve the ranking of both training and test observations. The clearest takeaway is that very small values of k , especially $k = 1$, give an overly optimistic picture of performance on the training data.

Increasing k makes predictions less dependent on any single training label and usually reduces their variability. However, using too many neighbors can smooth over useful local patterns. Neither test AUC nor the training–test gap is guaranteed to change steadily on a given split.

With 1NN, predicted probabilities are either 0 or 1, so the test ROC curve has just one point between its endpoints, and its AUC equals $(\text{sensitivity} + \text{specificity})/2$. The perfect training AUC comes from each observation using its own label, while the test AUC reflects predictions for unseen observations.

Question 3: Peer evaluation of Homework 5

You will receive two Homework 5 submissions to review. The instructor's GitHub account, **rqzhu-aide**, will open a **GitHub Issue** in the repository containing your Homework 5 submission. It will contain links to two other students' repositories, where you can find their submitted homework reports.

Open your repository's **Issues** tab on GitHub to find the assignment. You may ask AI to explain GitHub Issues or help you locate the assignment.

Choose **one specific mistake or item for improvement** from any question in either report.

1. **Identify the mistake or item for improvement.** State which report and question you selected, and briefly describe what is wrong or could be improved. Examples include incorrect code, a calculation error, a misinterpretation of results, a plot that could communicate the findings more clearly, or anything else you think could be done better.
2. **Explain your assessment.** Quote the relevant text, code, or figure caption exactly. Explain why the selected answer is incorrect or how it could be improved, referring to its reasoning, calculations, code, figures, or interpretation. **Complete this part yourself, without AI assistance.**
3. **Provide a correction.** Write a corrected or improved answer to the selected question, including any calculations, code, or figures needed.

After completing your review, **open a GitHub Issue in the repository containing the report you selected.** Identify the report and question, and include your assessment and a detailed revision plan explaining what the author should change and why. Include a link to your posted GitHub Issue in your Homework 6 answer.

I selected Question 4(b) from Jacob Jacobson's Homework 05. The discussion identifies common classification mistakes using raw counts. Adding percentages within each true digit class would make the comparison clearer because the test set contains different numbers of each digit. The report states?, "For both metrics, the most common single confusion is a true 2 predicted as 0 (seven Euclidean cases and six Manhattan cases)." Instead, using percentages would provide useful context but it's correct as a comparison of counts. If you reported both the counts and percentages, it would allow to distinguish the most common mistakes from those that affect a larger share of a digit class. For both distance measures, predicting a true 2 as 0 is the most frequent individual mistake by count: seven cases with Euclidean distance and six with Manhattan distance. These represent 14% and 12% of the 50 true 2s. By comparison, predicting a true 5 as 0 occurs four and five times, respectively, representing 15.4% and 19.2% of the 26 true 5s. Thus, a smaller error count can still affect a larger proportion of a digit class. Adding column-normalized percentages alongside the confusion-matrix counts would make these differences easier to interpret.

<https://github.com/jessicacudzich/stat432-fall2026/issues/2>